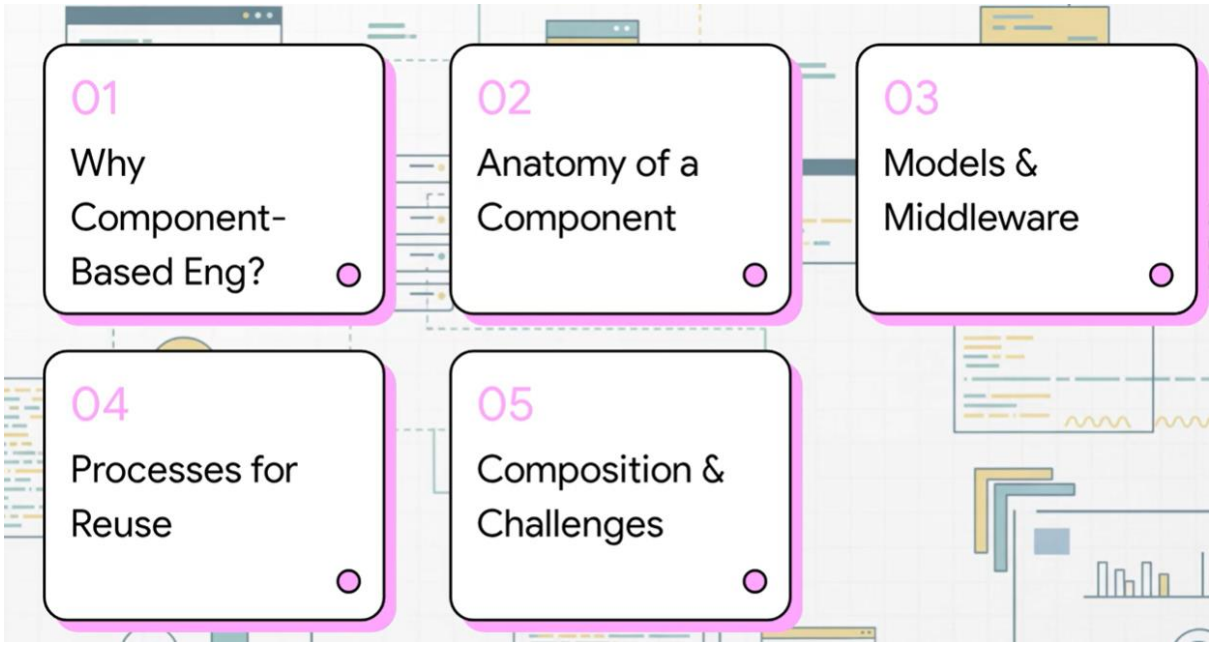
Lesson 1 Component Based software engineering fundamentals





Where we are in Ages of Software

1. Functional Programming — The Math Age (1950s–Present)

- **Motto:** Everything is a function.
- **Objective:** Predictable logic. Same input, same output.
- **Example:** A coffee grinder. Beans in, ground coffee out. No surprises.

2. OOP — The "Things" Age (1980s–1990s)

- **Motto:** Everything is an object.
- **Objective:** Model real-world things. Bundle data with behavior.
- **Example:** A custom-built car. Parts fit perfectly together, but you can't swap the engine.

3. CBSE — The Lego Age (1990s–2000s)

- **Motto:** Everything is a plug-in part.

- **Objective:** Build with independent, standardized pieces that snap together.
- **Example:** Lego bricks. Any piece connects to any other piece. Swap anytime.

4. Cloud Native — The City Grid Age (2010s–Present)

- **Motto:** Everything is a cloud service.
- **Objective:** Tiny services on a global grid. Scale instantly. One breaks, others survive.
- **Example:** Electricity. You plug in, it just works. You don't build the power plant.



The 4 Pillars of CBSE

1. Independent Components

- **Example:** Lego bricks.
    - *Each piece stands alone. You can pull one out and use it in a totally different toy without breaking anything.*

2. Component Standards

- **Example:** USB plugs.
    - *Everyone agrees on the same shape. If it fits the hole, it works — no matter who made it.*

3. Middleware

- **Example:** Electricity.
    - *You plug in a TV, and it just works. You don't care where the power comes from. The "middle" handles the connection.*

4. Development Process Geared for Reuse

- **Example:** Car parts.
    - *A steering wheel is built once and used in many cars. The whole team plans ahead to reuse parts, not build one-offs.*

Why CBSE is Modern Compared to Other Approaches — and What It Achieves

Traditional approaches (structured programming, monolithic design) build software as one large, tightly-coupled block of code written mostly from zero. CBSE is considered "modern" because it addresses the weaknesses of that model:

Traditional Approach	CBSE Approach	What CBSE Achieves
Code written from scratch for every project	Reuse of existing, tested components	Faster development, lower cost
Whole system tested together at the end	Components tested independently before assembly	Higher reliability, easier debugging
Hard to change one part without breaking others	Loosely coupled components with clear interfaces	Easier maintenance and upgrades
One team builds everything	Different vendors/teams build different components	Parallel development, specialization
Difficult to scale across large systems	Components can be added/replaced/scaled independently	Scalability and flexibility

In short, CBSE achieves: speed, reuse, reliability, flexibility, and reduced cost — which is why it underpins nearly all modern software (mobile apps, cloud systems, enterprise software).

2. Characteristics of Components

- Reusability — a component can be used in many different systems without modification.
- Independence (Self-contained) — a component can function and be understood on its own.
- Composability — components can be combined with other components to form larger systems.
- Encapsulation — internal implementation/logic is hidden; only the interface is exposed.

- Well-defined interfaces — a component publishes exactly what services it offers and what it requires.
- Language and platform independence — a component can often be used regardless of the programming language it was written in, as long as the interface contract is respected.
- Deployability — a component is a unit that can be deployed/installed as a whole (e.g. a .jar, .dll, or web service).
- Documented and standardized — comes with clear documentation describing usage.

> ***Analogy:*** *Think of a phone charger: it is reusable (works with many phones), independent (works without needing the phone's internal circuitry), and has a well-defined interface (the plug/socket shape) that hides how electricity is converted inside.*

The five definitions to memorize

1. **Component:** A self-contained, reusable software unit that provides specific functionality and can be deployed as part of a system.

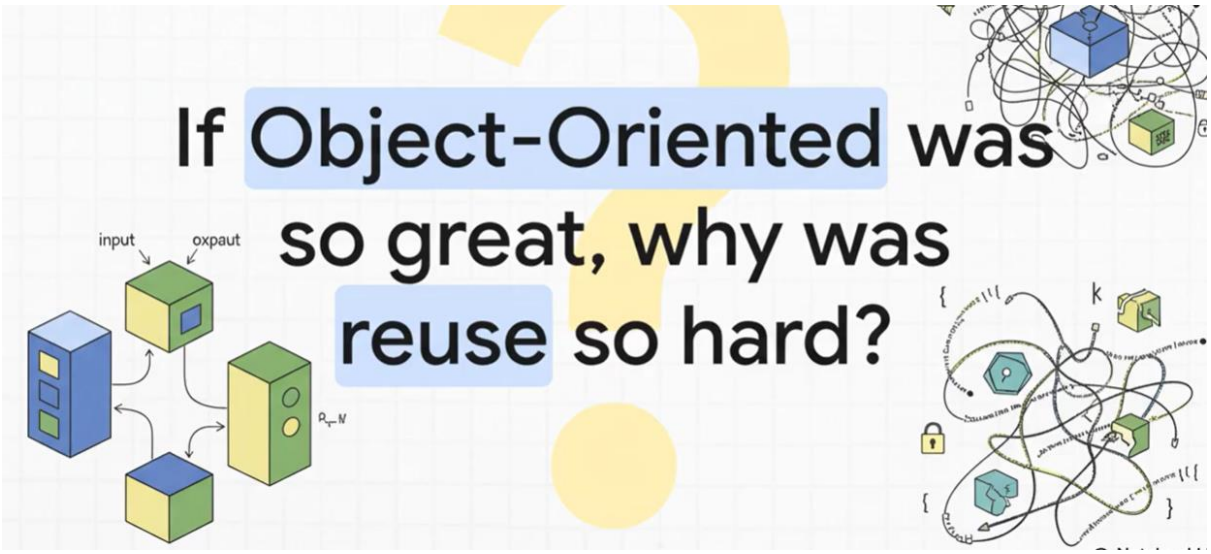
   Reusable software is software designed so it can be **used again in different applications or systems** without developing it from scratch each time.

   In CBSE, reuse is important because it:

   - **Saves development time** — use existing components instead of rebuilding them.
   - **Reduces cost** — less programming and testing effort.
   - **Reduces duplication** — the same functionality can serve multiple systems.
   - **Promotes modularity** — systems are divided into manageable components.
   - **Simplifies maintenance** — components can be updated or replaced independently.
   - **Can improve reliability** — previously tested components can be reused, although they still need evaluation.

2. **Interface:** A contract that defines how components communicate, including what a component **provides** and what it **requires**.
3. **Development FOR Reuse:** Creating components specifically so they can be reused in future systems.
4. **Development WITH Reuse:** Building a system by selecting and integrating components that already exist.

5. **Adaptation:** Modifying or wrapping an existing component so that its interface and behaviour fit the requirements of the new system.



The Paradox

- **Promise:** OOP should reuse code like Lego bricks. — *Good code should snap together and pull apart easily.*



- **Reality:** Code gets tangled, so developers copy-paste instead of reuse. — *When pieces are tied together by strings, pulling one out drags everything, so it's easier to just build a new one.*

Why It Fails

- **Inheritance Abuse:** Changing a parent class breaks child classes. —*Children are tied to the parent by strings; move the parent, and all children stumble.*
- **Tight Coupling:** Classes are hard-wired to specific tools (e.g., locked to one database). — *The class is tied to one specific tool, so you can't pull it out and use it with a different tool.*

- **God Object:** One massive class does everything. — *It's a giant knot of strings; you can't pull one string without unravelling the whole ball.*
- **Leaky Abstractions:** You must learn messy internal details just to reuse it. — *The strings are visible and messy; you can't just pick it up without untangling them first.*

1. Composition > Inheritance — The New Concept: "Has-a" Design

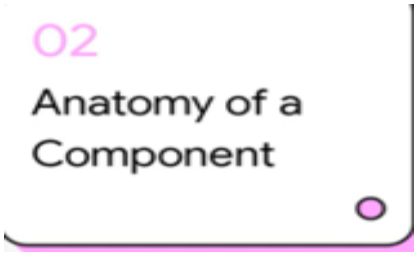
- **Basic OOP says:** Reuse through inheritance ("is-a"). A SportsCar *is a* Car.
- **The new idea:** Reuse through composition ("has-a"). A Car *has an* Engine.
- **Analogy:** *A phone case. Your phone has a case. You can swap it for a different color or style anytime without changing the phone itself. You are not stuck with one case forever just because your phone "is a" phone.*
- **Why it matters:** No permanent strings. Unplug and swap anytime.

2. Program to Interfaces — The New Concept: "Abstraction of Behavior"

- **Basic OOP says:** Create classes and call their methods directly.
- **The new idea:** Depend only on the *contract* (interface), never on the *concrete class*.
- **Analogy:** *A TV remote. The remote doesn't care if it's pointing at a Samsung, LG, or Sony TV. It just sends a signal, and any TV that understands the signal responds. The remote follows a universal contract.*
- **Why it matters:** Any device fits without needing to know what's inside.

3. Dependency Injection — The New Concept: "Inversion of Control"

- **Basic OOP says:** A class creates its own objects with new.
- **The new idea:** A class *receives* its tools from outside. It doesn't build them.
- **Analogy:** *A printer. Your computer doesn't build the printer. You plug in whatever printer you have — HP, Canon, or Epson — and the computer just sends the document. If you buy a new printer, the computer still works.*
- **Why it matters:** The class holds no strings to any specific tool, so it works in any context.



What Makes a True Software Component?

Characteristic	Description
Composable	All interactions happen through public interfaces.
Deployable	Self-contained and can operate as a stand-alone entity.
Documented	Fully documented so users can assess its suitability.
Independent	Can be deployed without using other specific components.
Standardized	Conforms to a standard component model.

What Makes a True Software Component?

1. Composable

- **What it means:** All interactions happen through public interfaces.
- You only talk to it through the front door. You don't need to know what's happening inside the house.
- **Example:** *A **TV remote**. You press the "Volume Up" button (the public interface). You don't need to know how the remote sends the signal or how the TV processes it. You just use the buttons.*

2. Deployable

- **What it means:** Self-contained and can operate as a stand-alone entity.
- It can work all by itself. You don't need to install a bunch of other stuff just to make it run.
- **Example:** *A **portable power bank**. You can take it anywhere and charge your phone without plugging it into a wall. It is self-contained.*

3. Documented

- **What it means:** Fully documented so users can assess its suitability.
- It comes with a clear instruction manual. You can read what it does and decide if it's the right tool for your job.
- **Example: IKEA furniture instructions**. *Before you buy or build, you read the manual to see if the table fits your room and if you have the right tools to assemble it.*

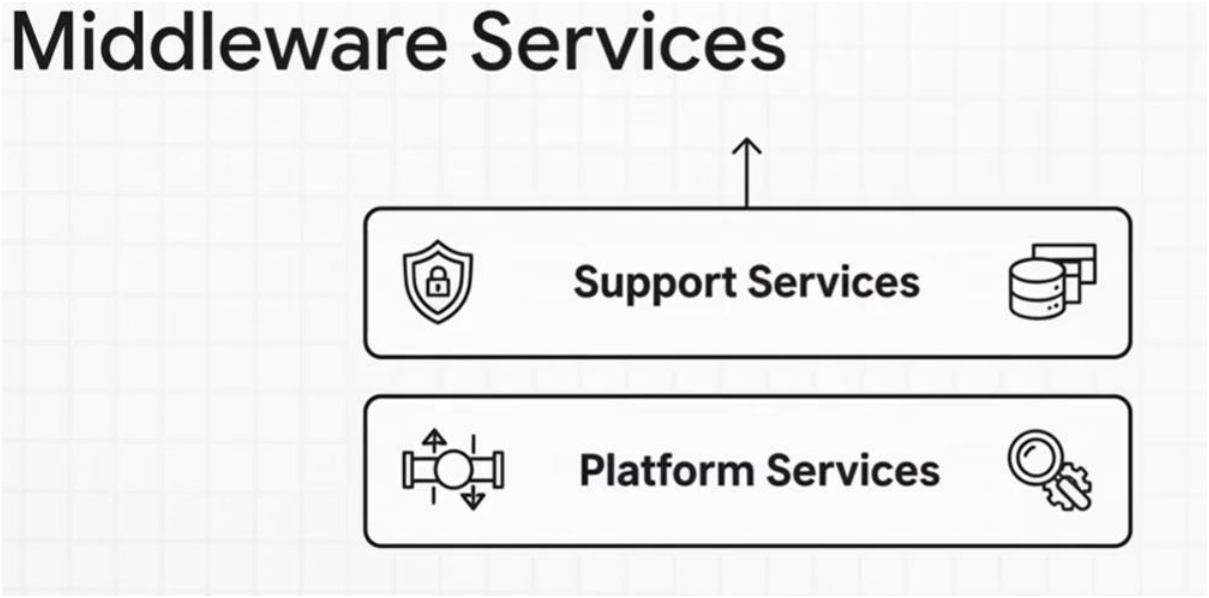
4. Independent

- **What it means:** Can be deployed without using other specific components.
- It doesn't play favorites. It doesn't require one specific brand of tool to work. It can work with anything that follows the rules.
- **Example:** *A **USB flash drive**. You can plug it into an HP laptop, a Dell desktop, or a Mac. It doesn't require a specific brand of computer to work.*

5. Standardized

- **What it means:** Conforms to a standard component model.
- It follows the agreed-upon rules so it can connect with other pieces.
- **Example:** *A **standard lightbulb socket**. Any lightbulb that follows the standard shape will screw into any lamp that follows the standard shape. No glue or tape needed.*





Why Middleware Services in CBSE

Middleware keeps components independent (no tangled strings), allows different languages to interoperate, and makes swapping parts easy. It's the glue that makes CBSE actually work.

The Problem: Components are built by different teams, in different languages, on different computers. If they talk directly, it creates a tangled mess.

The Solution: Middleware is the invisible bridge between components. They don't talk to each other—they talk to the middleware.

The Analogy: Electricity. *Your TV plugs into the wall. It doesn't care if power comes from solar or coal. The grid (middleware) handles the connection.*

The Two Types

1. Platform Services (The Plumbing)

- **What:** Routing, message translation, load balancing.
- **Example:** A post office. You drop a letter; it figures out the address and delivers it. You don't drive it yourself.

2. Support Services (The Safety Net)

- **What:** Security, transactions, database connections.

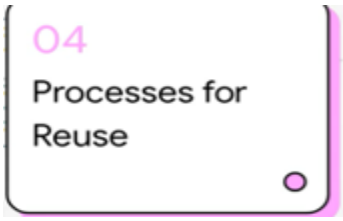
- **Example:** A bank teller. Verifies ID, records the transaction, ensures money doesn't vanish.

Real-World Example: Online Store

Components: Checkout (Python), Payment (Java), Inventory (C#).

Without Middleware: Checkout must know Java and C#. Swap PayPal for Stripe? Rewrite everything. Tangled mess.

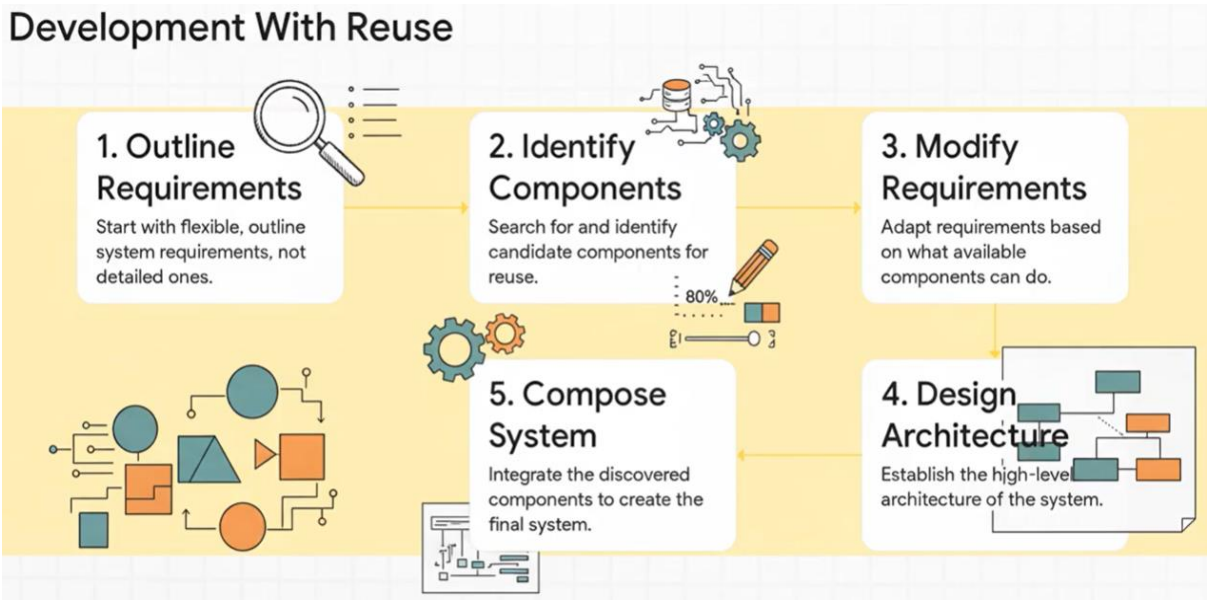
With Middleware: Checkout sends one standard message: "Charge $50." Middleware translates, secures, and routes it to Payment and Inventory. Swap PayPal for Stripe? Only update Payment. Checkout never changes.



Don't build everything from scratch. Stay flexible, find existing parts, adapt your plan to fit them, then assemble.







Development With Reuse

1. Outline Requirements

- **What:** Start with flexible, broad goals—not rigid, detailed specs.
- **Why:** Keeps the plan open so you can adapt to available pre-built parts later.

2. Identify Components

- **What:** Search for existing components that solve ~80% of your problem.
- **Why:** Writing code is expensive. Reusing tested parts saves time and reduces bugs.

3. Modify Requirements

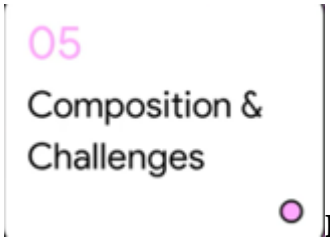
- **What:** Change your original plan to fit what the found components can actually do.
- **Why:** It's cheaper to adjust your goals than to rewrite a complex, tested component from scratch.

4. Design Architecture

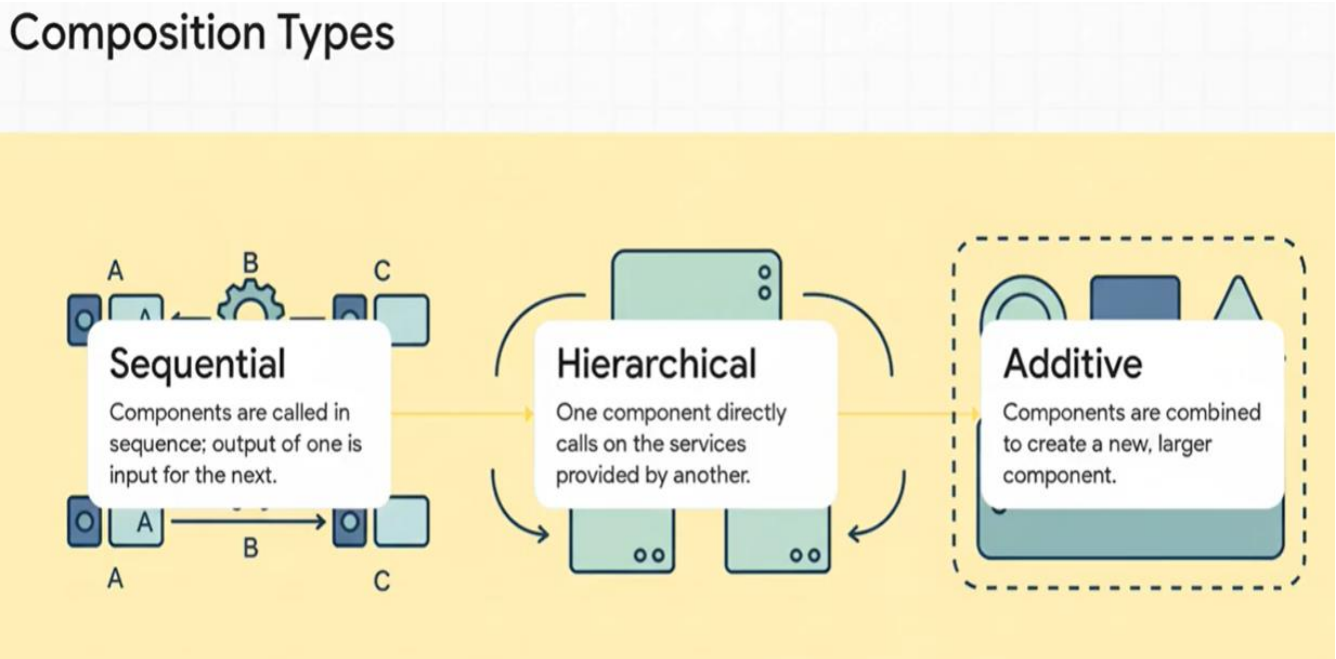
- **What:** Draw the high-level blueprint of how the selected components will connect.
- **Why:** You design the "glue" (connections), not the internals. This prevents a tangled mess.

5. Compose System

- **What:** Assemble the components into the final working system.
- **Why:** Since the heavy lifting is done, this is mostly assembly—making the system easier to test, maintain, and upgrade.



Putting all pieces together



Component Composition: The process of combining two or more independent components together, through their interfaces, to build a larger component or a complete system.

Composition techniques:

- Sequential composition — output of one component feeds into the input of the next.
- Hierarchical composition — a composite component is built from smaller sub-components.
- Additive composition — components are combined side-by-side to extend functionality.

> **Real-world example:** In an online shop, the "Cart" component, the "Payment" component, and the "Inventory" component are composed together to complete the checkout process.

Component Adaptation

Component Adaptation: The process of modifying a component (or building a wrapper around it) so that its interface matches the requirements of the system it is being integrated into, without changing its core internal logic.

Common adaptation techniques:

- Wrapping/Wrapper (Adapter pattern) — creating a new interface layer around the component to translate calls.
- White-box adaptation — modifying the component's actual source code (possible only if source is available).
- Black-box adaptation — using wrappers/adapters without touching internal source code (used for closed/proprietary components).

- Grey-box adaptation — limited modification allowed through extension points provided by the component vendor.

> ***Analogy:*** *Adaptation is like using a travel plug adapter: you don't rebuild your laptop charger (the component); you simply attach an adapter (wrapper) so its plug fits a foreign socket (the new system's interface).*

APIs (Application Programming Interfaces)

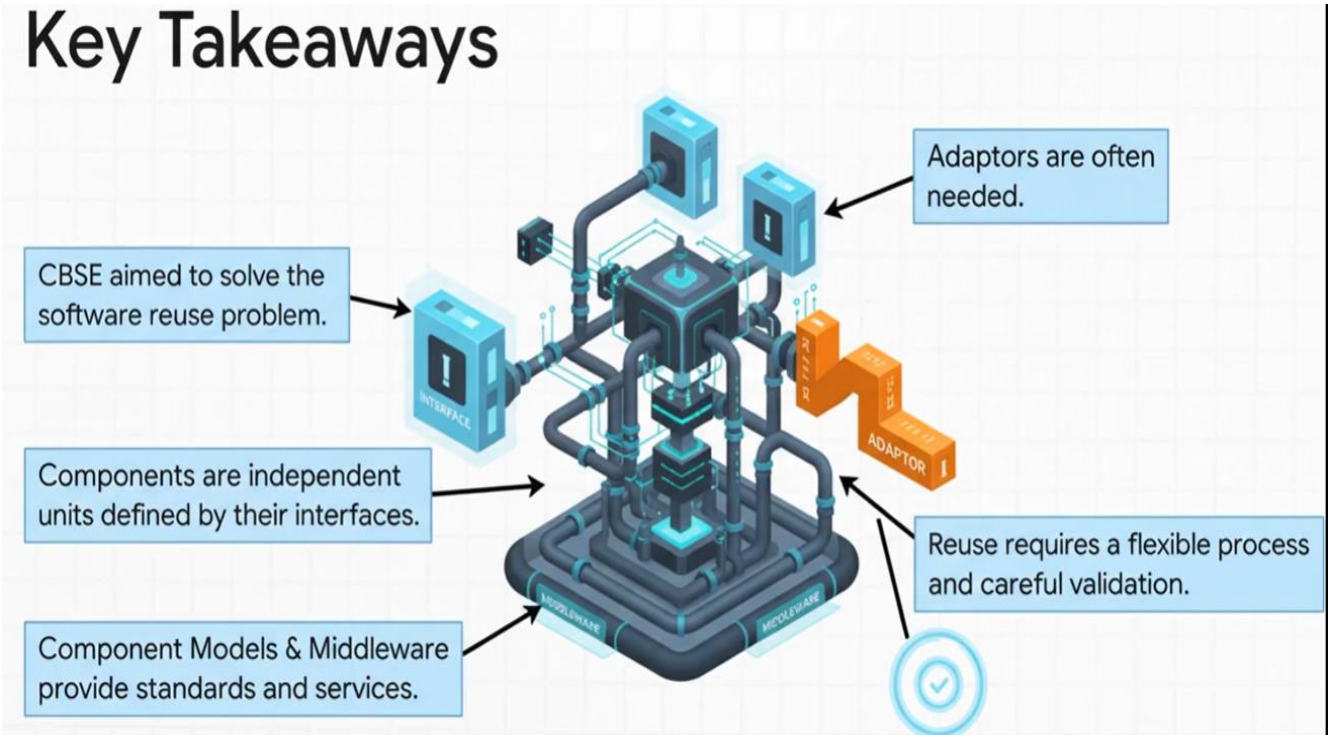
API: A defined set of rules, protocols, and tools that allows one software component or system to communicate with another. In CBSE, APIs are the practical realization of component interfaces.

APIs specify: what requests can be made, how to make them (syntax/format), what data formats to use, and what responses to expect.

```
// Example: calling a Weather API component (pseudocode)
GET https://api.weather.com/v1/current?city=Nairobi
Response: { "city": "Nairobi", "tempC": 24, "condition": "Sunny" }
```

> ***Analogy:*** *An API is like a restaurant menu and waiter combined: you (the client component) don't enter the kitchen (implementation) — you just place an order from the menu (interface) and the waiter (API) brings back the result.*

> **Real-world example:** Google Maps API lets any application embed maps and directions without building mapping software; M-Pesa Daraja API lets apps trigger mobile money payments without building telecom infrastructure.



Real-World Examples of CBSE

- Operating systems — Windows/Linux are composed of independently developed drivers, kernels, and utilities.
- Web browsers — built from components such as rendering engines, JavaScript engines, and plugins/extensions.
- E-commerce platforms — combine payment gateway components, shopping cart components, and recommendation engine components.
- Enterprise software (SAP, Salesforce) — modular components for HR, finance, inventory that can be enabled independently.
- Mobile apps — built using SDK components (Firebase for authentication, analytics, notifications).

Kenyan Applications of CBSE

> **Kenyan application:** M-Pesa and the Daraja API — Safaricom exposes M-Pesa functionality (STK push, C2B, B2C payments) as reusable API components that Kenyan businesses (e.g., Jumia, Uber, e-commerce sites) integrate into their own systems instead of building mobile money infrastructure themselves.
>
> **Kenyan application:** E-Citizen platform — the Kenyan government portal composes independent components/services (NTSA for driving licenses, Immigration for passports, Huduma for ID services, KRA for tax) under one unified access point.
>
> **Kenyan application:** Kenyan banking apps (e.g., KCB, Equity Bank mobile apps) — integrate third-party components such as biometric authentication modules, credit scoring engines, and payment switches (e.g., PesaLink) via APIs rather than building each from scratch.
>
> **Kenyan application:** University systems (e.g., student portals in Kenyan universities) — combine components for registration, fee payment (via M-Pesa API), and results/transcript management, often built or extended by local software firms using open-source components (e.g., Moodle plugins for e-learning).
>
> **Kenyan application:** Ride-hailing and delivery apps in Kenya (e.g., Little Cab, Bolt) — compose mapping components (Google Maps API), payment components (M-Pesa), and SMS/notification components (Africa's Talking API) into one working application.

Chapter 1 Summary

CBSE is a modern software engineering approach that builds systems by selecting, adapting, and composing reusable, independent components with well-defined interfaces, rather than writing all code from scratch. It relies on repositories to find components, APIs to connect them, and adaptation techniques to fit mismatched components. While it improves speed, cost, and reliability, it introduces challenges around integration, trust, and vendor dependency. In Kenya, CBSE is clearly

visible in systems like M-Pesa's Daraja API, e-Citizen, and mobile banking apps, which are built by composing independently developed components/services into unified platforms.